

{EPITECH}

NANOTEKSPICE

< DIGITAL ELECTRONICS />



Preliminaries



binary name: nanotekspice

language: C++

compilation: via Makefile, including re, clean and fclean rules or CMake

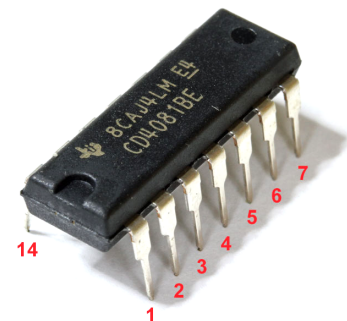


- ✓ The totality of your source files, except all useless files (binary, temp files, objfiles,...), must be included in your delivery.
- ✓ All the bonus files (including a potential specific Makefile) should be in a directory named bonus.
- ✓ Error messages have to be written on the error output, and the program should then exit with the 84 error code (0 if there is no error).

There are many programming languages: C, Lisp, Basic, APL, Intercal... Each has its specificities and may be efficient in a different way. They generally rely on a compiler to work. The compiler transforms code written in something similar to human language into a more primitive form. This primitive form is called **assembly language** when displayed in a human-readable format, or **machine language** when displayed under the format read by the microprocessor.

Much like there are many programming languages, there are many assembly languages: at least one per processor family, sometimes even one per single processor. Of course, knowing an assembly language for a microprocessor may not be enough to be efficient : processors aren't the only components in machines.

You may need to know how your machine works in order to create a useful assembly language: which address corresponds to the graphics card ? Which trap is assigned to this specific system call ? Hidden behind assembly and machine languages is the hardware itself. Hardware is built from digital electronic components: chipsets. Chipsets are tiny little functions with input and outputs that can be linked together to create more powerful functions, **just like in software**.

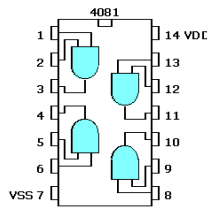


These functions are exclusively based on **boolean logic**, meaning the only inputs and outputs are `true` and `false`.

NanoTekSpice is a logic simulator that builds a graph (the nodes of which will be simulated digital electronic components, inputs or outputs) from a configuration file, and injects values into that graph to get results.

Chipsets

Here is an example of what's inside a simple chipset, the 4081 :



The 4081 is a little box that contains four AND gates, with two inputs and one output each. The 4081 features 14 connectors :

- ✓ pin 1 and pin 2 are inputs of one of the AND gates (gate 1),
- ✓ pin 3 is the output of gate 1,
- ✓ pin 4 is the output of gate 2,
- ✓ pin 5 and pin 6 are inputs of gate 2,
- ✓ pin 7 (VSS) is for electrical purposes; we will ignore it in this project,
- ✓ pin 8 and pin 9 are inputs of gate 3,
- ✓ pin 10 is the output of gate 3,
- ✓ pin 11 is the output of gate 4,
- ✓ pin 12 and pin 13 are inputs of gate 4,
- ✓ pin 14 (VOD) is for electrical purposes; we will ignore it in this project.

Here is the AND, OR, and XOR gate's **truth table** :

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Note this is exactly equivalent to the following function for the AND gate:

```
bool and_gate(bool a, bool b)
{
    return a && b;
}
```

There are plenty of chipsets. Some are primitive, like the 4081, while others represent more complicated functions which perform operations such as counting or serialising. Microprocessors are one of the more complicated family of chipsets as they provide interpreters.

Of course, microprocessors are composed of simpler chipsets, **which are themselves composed of chipsets** like the 4081. Everything boils down to boolean logic.

The Undefined State

In boolean algebra, there are only two values: `true` and `false`. In the electronic world, `true` means *VCC volt*. VCC is the power for the system, generally 5 volts. `false`, in most cases, means 0 volts.

There is however, another case we have to take into account : what if a component `A` needs to know a value `B` to compute its output, but `B` hasn't been defined yet?

`B` cannot be set to `true` or `false`, as that would be guessing its value and could mean the rest of the system receives the wrong value. The only solution is to say that `B` is `undefined`, a third state. This means you won't be using pure boolean algebra, and will have to integrate an undefined state.



Bye bye `bool`, welcome `enum` !

Here is the AND, OR and XOR gate's **truth table** including the `undefined` state:

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
0	U	0	U	U
1	0	0	1	1
1	1	1	1	0
1	U	U	1	U
U	0	0	U	U
U	1	U	1	U
U	U	U	U	U

The Configuration File

Here is a sample configuration file. It contains a graph description.

```
#three inputs and gate
.chipsets:
input    i0
input    i1
input    i2
4081     and0
output   out

.links:
i0:1     and0:1
i1:1     and0:2

and0:3   and0:5
i2:1     and0:6

and0:4   out:1
```

The **first part**, starting with the `".chipsets:"` statement is used to declare and name components that will be used by the program.

The **second part**, starting with the `".links:"` statement is used to declare links between components. Each link indicates which pin of which component are linked. Links are **bidirectional**.



White spaces between keywords on a line may be spaces or tabs. Statements are newline-terminated.

Comments start with a `'#'` and end with a newline. A comment can be at the start of a line or after an instruction.

Special Components

Here are a few special components, they are described in the provided files :

- ✓ `input` : a component with a single pin directly linked to the command line. Its value is initialized to `undefined`.
- ✓ `clock` : a component that works like an `input`, except its value is inverted after each simulation.
- ✓ `true` : a component with a single pin that is always `true`.
- ✓ `false` : a component with a single pin that is always `false`.
- ✓ `output` : a component with a single pin used as the output of a circuit.

Errors

When one of the following cases happens, **NanoTekSpice** must raise an exception and stop program execution neatly. Raising scalar exceptions is forbidden. Moreover, exception classes must inherit from `std::exception`.

- ✓ The circuit file includes one or several lexical or syntactic errors.
- ✓ A component type is unknown.
- ✓ A component name is unknown.
- ✓ Several components share the same name.
- ✓ No chipsets in the circuit.



There may be other errors. However, your simulator must never crash (uncaught exception, segmentation fault, bus error, infinite loop...) !

Execution

Your simulator must be able to run with a circuit file passed as parameter. The simulator also reads the standard input for the following commands:

- ✓ `exit` : closes the program.
- ✓ `display` : prints the current tick and the value of all inputs and outputs the standard output, each sorted by name in ASCII order.
- ✓ `input=value` : changes the value of an `input`. Possible `value` are 0, 1 and U. This also apply to `clocks`.
- ✓ `simulate` : simulate a tick of the circuit.
- ✓ `loop` : continuously runs the simulation (`simulate, display, simulate, ...`) without displaying a prompt, until `SIGINT (CTRL+C)` is received.

Feel free to add commands for debug.

Between commands, your program must display "> " as a prompt ('>' followed by a space).

When printing a value, if it is undefined, your program must display "U" (without the quotes).

When it reaches the end of its standard input (ever heard of `CTRL+D`?), the program must stop with status 0.

Here are some sample executions:

```
Terminal
$> cat -e or_gate.nts
.chipsets:$
input a$
input b$
4071 or$
output s$
.links:$
a:1 or:1$
b:1 or:2$
or:3 s:1$
$> ./nanotekspice or_gate.nts
> b=0
> a=1
> simulate
> display
tick: 1
input(s):
  a: 1
  b: 0
output(s):
  s: 1
> exit
$> echo $?
0
```

```
Terminal
$> cat -e bad.nts
.chipsets:$
input i$
output o$
.links:$
a:1 o:1$
$> ./nanotekspice bad.nts
Unknow component name 'a'.
$> echo $?
84
```



The error message is provided as an example. Feel free to use yours instead.


```
Terminal
$> cat -e clock.nts
.chipsets:$
clock cl$
output out$
.links:$
out:1 cl:1$
$> ./nanotekspice clock.nts
> display
tick: 0
input(s):
  cl: U
output(s):
  out: U
> cl=0
> display
tick: 0
input(s):
  cl: U
output(s):
  out: U
> simulate
> display
tick: 1
input(s):
  cl: 0
output(s):
  out: 0
> simulate
> display
tick: 2
input(s):
  cl: 1
output(s):
  out: 1
> simulate
> simulate
> simulate
> display
tick: 5
input(s):
  cl: 0
output(s):
  out: 0
> (CTRL+D) $> echo $?
0
```

Technical considerations

In order to help you through your implementation, we have designed the following instructions which you must follow. It is important that you understand why each of these instructions exists. We are not providing them for fun or to torture you!

The IComponent interface

Each of your components **MUST** implement a `nts::IComponent` interface. Here is an example :

```
namespace nts
{
    enum Tristate {
        Undefined = (-true),
        True = true,
        False = false
    };

    class IComponent
    {
    public:
        virtual ~IComponent() = default;

        virtual void simulate(std::size_t tick) = 0;
        virtual nts::Tristate compute(std::size_t pin) = 0;
        virtual void setLink(std::size_t pin, nts::IComponent &other, std::size_t otherPin) = 0;
    };
}
```

You are free to modify it, but be aware it's already *perfect*.

Here is a list of the components you must implement. Manuals for gates and advanced components are provided alongside this subject. A `.nts` file is provided for each component, use them for your tests.

- ✓ Special components : `input`, `output`, `true`, `false`, `clock`.
- ✓ Elementary components:
 - `and` : single AND gate.
 - `or` : single OR gate.
 - `xor` : single XOR gate.
 - `not` : single INVERTER gate.
- ✓ Gates components :
 - 4001 : Four NOR gates.
 - 4011 : Four NAND gates.
 - 4030 : Four XOR gates.
 - 4069 : Six INVERTER gates.
 - 4071 : Four OR gates.
 - 4081 : Four AND gates.
- ✓ Advanced components :
 - 4008 : 4 bits adder.
 - 4013 : Dual Flip Flop.
 - 4017 : 10 bits Johnson decade.
 - 4040 : 12 bits counter.
 - 4094 : 8 bits shift register.
 - 4512 : 8 channel data selector.
 - 4514 : 4 bits decoder.
 - 4801 : Random access memory.
 - 2716 : Read only memory (memory initilized from `./rom.bin`).

You must also implement the `logger` component. It appends the character computed from its inputs to the file `./log.bin` on the positive-going edge of the `clock` when `inhibit` is false. More details in the provided NTS files.



It is FORBIDDEN to manipulate pointers or references to these classes in your graph. You must **ONLY** manipulate pointers and references to `nts::IComponent`.

Creation of a new `nts::IComponent`

You must write a member function of a class **relevant to your system** that will let you create new components in a generic manner. This function must have the following prototype:

```
std::unique_ptr<nts::IComponent> createComponent(const std::string &type);
```

This function creates a new `nts::IComponent` according to its `type` parameter. To do so, it could call one of the following `private` member functions:

```
std::unique_ptr<nts::IComponent> create4001() const;  
std::unique_ptr<nts::IComponent> create4013() const;  
std::unique_ptr<nts::IComponent> create4040() const;  
// etc...
```

However, there are other techniques you may use to implement this **factory** (ever heard of **lambdas**?).

Bonus

This project is extensible with various bonuses. Here are a few examples:

- ✓ Graphic output of the graph. Feel free to add a `.graphic` section to the configuration file with graphical information.
- ✓ Graphic component *"7seg"*. Uses 4 pins as inputs. Displays a hexadecimal number matching its input.
- ✓ Graphic component *"matrix"*. Uses 8 pins as a color shade and 12 bits as pixel selections. Outputs a 64x64 pixel picture.
- ✓ i4004 support (Or any other microprocessor, possibly one of your own design. Must be Turing complete).
- ✓ Simulating a little computer.

Feel free to use SFML, SDL, Allegro or any other library for 2D or 3D graphics.

v 3.1.2

{EPITECH}